



CYBERSNOVA CYBERSECURITY INTERNSHIP PROJECT REPORT

Name:	Sameena K D
Email:	skdhalayat@gmail.com
Reference No:	CSN-SP-OL-26212
Registration No:	CSN-SP-2026-212
Domain:	Cyber Security
Project Level:	Beginner Level – Advance Level



Sl.No	Project Description (BEGINNER LEVEL)	Pg.No
01	Network Reconnaissance & Service Enumeration ➤ In this project, I performed network reconnaissance and service enumeration on an authorized practice website. ➤ I used Nmap, WhatWeb, and Netcat to find open ports, available services, and the technologies used by the website. ➤ I also recorded the findings and screenshots as part of the project.	4-10
02	Hidden Directory & File Enumeration ➤ In this project, I performed hidden directory and file enumeration on an authorized vulnerable website. ➤ I used Gobuster, Dirsearch, and Dirb to discover hidden or unlinked directories and files available on the web server. ➤ I recorded the discovered resources, analyzed their potential security risks, and captured screenshots as part of the project.	11-23
03	Login Traffic Analysis • In this project, I captured and analyzed the network traffic generated during a login session on an authorized vulnerable website. • I used Wireshark and Burp Suite to inspect the HTTP requests and identify how login information was transmitted. • I analyzed the captured traffic and documented the security weakness along with screenshots and recommendations.	24-27
04	Basic Web Vulnerability Assessment • In this project, I performed a basic security assessment of an authorized vulnerable website. • I used Nikto and Burp Suite to identify common web security weaknesses and inspect HTTP traffic. • I documented the identified vulnerabilities, their severity, possible impact, and recommended mitigation techniques.	28-34



Sl.No	Project Description (ADVANCED LEVEL)	Pg.No
01	Complete web application penetration test • In this project, I performed a web application penetration test on an authorized local vulnerable application. • I used Nmap and manual testing to identify and verify vulnerabilities such as SQL Injection, Reflected XSS, Command Injection, and Local File Inclusion. • I documented the vulnerabilities, their security impact, screenshots, and recommended mitigation techniques.	35-47



BEGINNER LEVEL

PROJECT 1

NETWORK RECONNAISSANCE & SERVICE ENUMERATION

1. Objective

The objective of this project was to perform basic reconnaissance on an authorized vulnerable web application.

The main purpose was to:

- Check whether the target was reachable.
- Identify the IP address of the target.
- Scan the target for open ports.
- Identify the services running on the open ports.
- Identify the technologies used by the website.
- Perform basic TCP connectivity testing.
- Record the observations and results.

2. Target

Target Website: <http://testasp.vulnweb.com/>

3. Tools Used

- Nmap
- WhatWeb
- Netcat



4. Methodology

The following steps were performed during the reconnaissance process:

1. Checked the connectivity of the target.
2. Performed a basic Nmap scan.
3. Identified the IP address of the target.
4. Scanned the target for open ports.
5. Performed service and version detection.
6. Used WhatWeb to identify website technologies.
7. Used Netcat to test TCP connectivity.
8. Recorded the results and captured screenshots as evidence.

Nmap Basic Scanning

I started the reconnaissance process by performing a basic Nmap scan on the target.

Command used: nmap testasp.vulnweb.com

Figure 1: Nmap basic scan of testasp.vulnweb.com

```
(sameena@kali)-[~]
$ nmap testasp.vulnweb.com
Starting Nmap 7.95 ( https://nmap.org ) at 2026-09-26 12:02 IST
Nmap scan report for testasp.vulnweb.com (44.238.29.244)
Host is up (0.022s latency).
Other addresses for testasp.vulnweb.com (not scanned): 64:ff9b::2cee:1df4
rDNS record for 44.238.29.244: ec2-44-238-29-244.us-west-2.compute.amazonaws.com
Not shown: 999 filtered tcp ports (no-response)
PORT      STATE SERVICE
80/tcp    open  http

Nmap done: 1 IP address (1 host up) scanned in 17.94 seconds
```

Finding:

Port 80/tcp was found to be open, indicating that an HTTP service is accessible on the target.



Nmap Service & Version Enumeration

-sV command asks Nmap to determine what service and version are running on an open port.

Command used: nmap -sV testasp.vulnweb.com

Figure 2:

```
(sameena@kali)-[~]
$ nmap -sV testasp.vulnweb.com
Starting Nmap 7.95 ( https://nmap.org ) at 2026-09-26 12:03 IST
Nmap scan report for testasp.vulnweb.com (44.238.29.244)
Host is up (0.020s latency).
Other addresses for testasp.vulnweb.com (not scanned): 64:ff9b::2cee:1df4
rDNS record for 44.238.29.244: ec2-44-238-29-244.us-west-2.compute.amazonaws.com
Not shown: 999 filtered tcp ports (no-response)
PORT      STATE SERVICE VERSION
80/tcp    open  http      Microsoft IIS httpd 8.5
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 18.42 seconds
```

Finding:

The target has an HTTP web service running on TCP port 80, and Nmap identifies the web server as Microsoft IIS 8.5.

Nmap	-sV Nmap
Nmap mainly checks which ports are open, closed, or filtered on the target.	Nmap -sV goes one step further and tries to identify the service and its version running on an open port.
Example: Nmap shows Port 80 → HTTP	Example: -sV identified HTTP → Microsoft IIS 8.5
Its basically like finding the door	Its basically like finding out what is behind the door



WhatWeb

It is used to identify the technologies used by website.

Command used: whatweb http://testasp.vulnweb.com/

Figure 3:

```
(sameena@kali)-[~]  
$ whatweb http://testasp.vulnweb.com  
http://testasp.vulnweb.com [200 OK] ASP.NET, Cookies[ASPSESSIONIDCCBQRCSA], Country[UNITED STATES][US], HTTPServer[Microsoft-IIS/8.5], IP[44.238.29.244], Microsoft-IIS[8.5], Title[acuforum forums], X-Powered-By[ASP.NET]  
  
(sameena@kali)-[~]  
$
```

Finding:

WhatWeb identified the target's web server, application technology, session cookie, and application title.



Netcat

It is a networking tool that can create connections between computers using TCP or UDP.

Command used: nc testasp.vulnweb.com 80

HTTP Request Used: GET / HTTP/1.1

Host: testasp.vulnweb.com

Connection: close

Figure 4:

```
sameena@kali: ~  
(sameena@kali)-[~]  
$ nc testasp.vulnweb.com 80  
GET / HTTP/1.1  
Host: testasp.vulnweb.com  
Connection: close  
  
HTTP/1.1 200 OK  
Cache-Control: private  
Content-Type: text/html  
Server: Microsoft-IIS/8.5  
Set-Cookie: ASPSESSIONIDCCBQRCSA=NNELKJNDKHOPAEIFGBODAEHP; path=/  
X-Powered-By: ASP.NET  
Date: Sat, 26 Sep 2026 15:17:55 GMT  
Connection: close  
Content-Length: 3538
```



```
Content-Length: 3538

<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd">
<html><!-- InstanceBegin template="/Templates/MainTemplate.dwt.asp" codeOutsideHTMLOutsideIsLocked="false" -->
<head>
<!-- InstanceBeginEditable name="doctitle" -->
<title>acuforum forums</title>
<!-- InstanceEndEditable -->
<meta http-equiv="Content-Type" content="text/html; charset=iso-8859-1">
<!-- InstanceBeginEditable name="head" -->
<!-- InstanceEndEditable -->
<link href="stylesheet" rel="stylesheet" type="text/css">
</head>
<body>
<table width="100%" border="0" cellpadding="10" cellspacing="0">
  <tr bgcolor="#008F00">
    <td width="306px"><a href="https://www.acunetix.com/"></a></td>
    <td align="right" valign="middle" bgcolor="#008F00" class="disclaimer">TEST and Demonstration site for <a href="https://www.acun
etix.com/vulnerability-scanner/">Acunetix Web Vulnerability Scanner</a></td>
  </tr>
  <tr>
    <td colspan="2"><div class="menubar"><a href="Templatize.asp?item=html/about.html" class="menu">about</a> - <a href="Default.asp
" class="menu">forums</a> - <a href="Search.asp" class="menu">search</a>
      - <a href="./Login.asp?RetURL=%2FDefault%2Easp%3F" class="menu">login</a> - <a href="./Register.asp?RetURL=%2FDefault%2Easp%3F"
class="menu">register</a>
      - <a href="https://www.acunetix.com/vulnerability-scanner/sql-injection/" class="menu">SQL scanner</a> - <a href="https://ww
w.acunetix.com/websitesecurity/sql-injection/" class="menu">SQL vuln help</a>
    </div></td>
  </tr>
  <tr>
    <td colspan="2"><!-- InstanceBeginEditable name="MainContentLeft" -->
      <table width="100%" border="0" cellspacing="0" cellpadding="5">
        <tr>
          <td class="tableheader">Forum</td>
          <td class="tableheader">Threads</td>
          <td class="tableheader">Posts</td>
          <td class="tableheader">Last Post</td>
        </tr>
        <tr bgcolor="#FFFFFF"><td><div class='forumtitle'><a href='showforum.asp?id=0'>Acunetix Web Vulnerability Scanner</a></div><
div class='forumdescription'>Talk about Acunetix Web Vulnerability Scanner</div></td><td>5</td><td>5</td><td>11/9/2005 1:45:54 PM</td>
</tr><tr bgcolor="#e9ffe9"><td><div class='forumtitle'><a href='showforum.asp?id=1'>Weather</a></div><div class='forumdescription'>
What weather is in your town right now</div></td><td>1</td><td>1</td><td>11/9/2005 12:16:35 PM</td></tr><tr bgcolor="#FFFFFF"><td><d
iv class='forumtitle'><a href='showforum.asp?id=2'>Miscellaneous</a></div><div class='forumdescription'>Anything crossing your mind
can be posted here</div></td><td>0</td><td>0</td><td></td></tr>
      </table><!-- InstanceEndEditable --></td>
    <td align="right" bgcolor="#FFFFFF">
      <td colspan="2" class="footer">Copyright 2019 Acunetix Ltd.</td>
    </tr>
  </table>
<div style="background-color:lightgray;width:80%;margin:auto;text-align:center;font-size:12px;padding:1px">
  <p style="padding-left:20%;padding-right:20%"><b>Warning</b>: This forum is deliberately vulnerable to SQL Injections, direc
tory traversal, and other web-based attacks. It is built using ASP and it is here to help you test Acunetix. The entire content of t
he forum is erased daily. All the posts are real-life examples of how attackers are trying to break into insecure web applications.
Please be careful and do not follow links that are posted by malicious parties.</p>
</div>
</body>
<!-- InstanceEnd --></html>
```



Finding

Netcat provided direct information about the HTTP response and confirmed details identified by Nmap and WhatWeb. It also revealed additional application details and accessible paths that can be useful for further reconnaissance.

Conclusion

This project helped me understand the basic process of network reconnaissance and service enumeration. By using Nmap, WhatWeb, and Netcat, I was able to gather information about the authorized practice target, identify an open HTTP service, and understand how different reconnaissance tools are used during a security assessment.



PROJECT 2

HIDDEN DIRECTORY & FILE ENUMERATION

1. Objective

The objective of this project was to find hidden directories and files present on an authorized vulnerable web application.

The main purpose was to:

- Find directories and files that are not directly visible on the website.
- Check the HTTP status of the discovered resources.
- Verify some of the discovered resources.
- Understand how directory enumeration is useful during reconnaissance

2. Target

Target Website: <http://testasp.vulnweb.com/>

3. Tools Used

- Gobuster
- Dirsearch
- Dirb



Methodology

The following steps were performed:

1. Accessed the authorized vulnerable target.
2. Selected the common.txt wordlist.
3. Performed directory enumeration using Gobuster.
4. Performed directory enumeration using Dirsearch.
5. Performed directory enumeration using Dirb.
6. Recorded the discovered directories and files.
7. Checked the response codes of interesting resources.
8. Compared the results from all three tools.
9. Captured screenshots as evidence.

Gobuster

For this project, Gobuster was used to discover directories and files that were not directly visible from the main webpage.

Directory Enumeration Mode

- The dir mode was checked using: **gobusterdir**

The output confirmed that the url and wordlist parameters are required for the scan.

The dir mode is used to search for directories and files on a web server.



```
sameena@kali: ~  
  
(sameena@kali)-[~]  
$ gobuster  
NAME:  
gobuster - the tool you love  
  
USAGE:  
gobuster command [command options]  
  
VERSION:  
3.8  
  
AUTHORS:  
Christian Mehlmauer (@firefart)  
OJ Reeves (@TheColonial)  
  
COMMANDS:  
dir      Uses directory/file enumeration mode  
vhost    Uses VHOST enumeration mode (you most probably want to use the IP address as the URL parameter)  
dns      Uses DNS subdomain enumeration mode  
fuzz     Uses fuzzing mode. Replaces the keyword FUZZ in the URL, Headers and the request body  
tftp     Uses TFTP enumeration mode  
s3       Uses aws bucket enumeration mode  
gcs      Uses gcs bucket enumeration mode  
help, h  Shows a list of commands or help for one command  
  
GLOBAL OPTIONS:  
--help, -h      show help  
--version, -v   print the version  
  
(sameena@kali)-[~]  
$ gobuster dir  
NAME:  
gobuster dir - Uses directory/file enumeration mode  
  
USAGE:  
gobuster dir [command options] [arguments...]  
  
OPTIONS:  
--url value, -u value           The target URL  
--cookies value, -c value       Cookies to use for the requests  
--username value, -U value      Username for Basic Auth  
--password value, -P value      Password for Basic Auth  
--follow-redirect, -r           Follow redirects (default: false)  
--headers value, -H value [ --headers value, -H value ] Specify HTTP headers, -H 'Header1: val1' -H 'Header2: val2'  
--no-canonicalize-headers, --nch Do not canonicalize HTTP header names. If set header names are sent as i  
s (default: false)  
--method value, -m value        the password to the p12 file (default: "GET")  
--useragent value, -a value      Set the User-Agent string (default: "gobuster/3.8")  
--random-agent, --rua           Use a random User-Agent string (default: false)  
--proxy value                   Proxy to use for requests [http(s)://host:port] or [socks5://host:port]  
--timeout value, --to value     HTTP Timeout (default: 10s)  
--no-tls-validation, -k         Skip TLS certificate verification (default: false)  
--retry                         Should retry on request timeout (default: false)  
--retry-attempts value, --ra value Times to retry on request timeout (default: 3)  
--client-cert-pem value, --ccp value public key in PEM format for optional TLS client certificates  
--client-cert-pem-key value, --ccpk value private key in PEM format for optional TLS client certificates (this key  
needs to have no password)  
--client-cert-p12 value, --ccp12 value a p12 file to use for options TLS client certificates  
--client-cert-p12-password value, --ccp12p value the password to the p12 file  
--tls-renegotiation             Enable TLS renegotiation (default: false)  
--interface value, --iface value specify network interface to use. Can't be used with local-ip  
--local-ip value               specify local ip of network interface to use. Can't be used with interfa  
ce
```



```
sameena@kali: ~  
NAME:  
  gobuster dir - Uses directory/file enumeration mode  
  
USAGE:  
  gobuster dir [command options] [arguments...]  
  
OPTIONS:  
  --url value, -u value           The target URL  
  --cookies value, -c value       Cookies to use for the requests  
  --username value, -U value      Username for Basic Auth  
  --password value, -P value      Password for Basic Auth  
  --follow-redirect, -r           Follow redirects (default: false)  
  --headers value, -H value [ --headers value, -H value ] Specify HTTP headers, -H 'Header1: val1' -H 'Header2: val2'  
  --no-canonicalize-headers, --nch Do not canonicalize HTTP header names. If set header names are sent as i  
s (default: false)  
  --method value, -m value        the password to the p12 file (default: "GET")  
  --useragent value, -a value      Set the User-Agent string (default: "gobuster/3.8")  
  --random-agent, --rua           Use a random User-Agent string (default: false)  
  --proxy value                  Proxy to use for requests [http(s)://host:port] or [socks5://host:port]  
  --timeout value, --to value     HTTP Timeout (default: 10s)  
  --no-tls-validation, -k         Skip TLS certificate verification (default: false)  
  --retry                        Should retry on request timeout (default: false)  
  --retry-attempts value, --ra value Times to retry on request timeout (default: 3)  
  --client-cert-pem value, --ccp value public key in PEM format for optional TLS client certificates]  
  --client-cert-pem-key value, --ccpk value private key in PEM format for optional TLS client certificates (this key  
needs to have no password)  
  --client-cert-p12 value, --ccp12 value a p12 file to use for options TLS client certificates  
  --client-cert-p12-password value, --ccp12p value the password to the p12 file  
  --tls-renegotiation            Enable TLS renegotiation (default: false)  
  --interface value, --iface value specify network interface to use. Can't be used with local-ip  
  --local-ip value              specify local ip of network interface to use. Can't be used with interfa  
ce  
  --wordlist value, -w value      Path to the wordlist. Set to - to use STDIN.  
  --delay value, -d value         Time each thread waits between requests (e.g. 1500ms) (default: 0s)  
  --threads value, -t value       Number of concurrent threads (default: 10)  
  --wordlist-offset value, --wo value Resume from a given position in the wordlist (default: 0)  
  --output value, -o value        Output file to write results to (defaults to stdout)  
  --quiet, -q                    Don't print the banner and other noise (default: false)  
  --no-progress, --np            Don't display progress (default: false)  
  --no-error, --ne               Don't display errors (default: false)  
  --pattern value, -p value       File containing replacement patterns  
  --discover-pattern value, --pd value File containing replacement patterns applied to successful guesses  
  --no-color, --nc               Disable color output (default: false)  
  --debug                        enable debug output (default: false)  
  --status-codes value, -s value  Positive status codes (will be overwritten with status-codes-blacklist i  
f set). Can also handle ranges like 200,300-400,404  
  --status-codes-blacklist value, -b value Negative status codes (will override status-codes if set). Can also hand  
le ranges like 200,300-400,404. (default: "404")  
  --extensions value, -x value   File extension(s) to search for  
  --extensions-file value, -X value Read file extension(s) to search from the file  
  --expanded, -e                 Expanded mode, print full URLs (default: false)  
  --no-status, -n                Don't print status codes (default: false)  
  --hide-length, --hl            Hide the length of the body in the output (default: false)  
  --add-slash, -f                Append / to each request (default: false)  
  --discover-backup, --db        Upon finding a file search for backup files by appending multiple backup  
extensions (default: false)  
  --exclude-length value, --xl value exclude the following content lengths (completely ignores the status). Y  
ou can separate multiple lengths by comma and it also supports ranges like 203-206  
  --force                        Continue even if the prechecks fail. Please only use this if you know wh  
at you are doing, it can lead to unexpected results. (default: false)  
  --help, -h                     show help  
2026/09/26 21:41:03 Required flags "url, wordlist" not set  
  
sameena@kali)~$
```



```
sameena@kali: ~  
--no-error, --ne Don't display errors (default: false)  
--pattern value, -p value File containing replacement patterns  
--discover-pattern value, --pd value File containing replacement patterns applied to successful guesses  
--no-color, --nc Disable color output (default: false)  
--debug enable debug output (default: false)  
--status-codes value, -s value Positive status codes (will be overwritten with status-codes-blacklist i  
f set). Can also handle ranges like 200,300-400,404  
--status-codes-blacklist value, -b value Negative status codes (will override status-codes if set). Can also hand  
le ranges like 200,300-400,404. (default: "404")  
--extensions value, -x value File extension(s) to search for  
--extensions-file value, -X value Read file extension(s) to search from the file  
--expanded, -e Expanded mode, print full URLs (default: false)  
--no-status, -n Don't print status codes (default: false)  
--hide-length, --hl Hide the length of the body in the output (default: false)  
--add-slash, -f Append / to each request (default: false)  
--discover-backup, --db Upon finding a file search for backup files by appending multiple backup  
extensions (default: false)  
--exclude-length value, --xl value exclude the following content lengths (completely ignores the status). Y  
ou can separate multiple lengths by comma and it also supports ranges like 203-206  
--force Continue even if the prechecks fail. Please only use this if you know wh  
at you are doing, it can lead to unexpected results. (default: false)  
--help, -h show help  
2026/09/26 21:41:03 Required flags "url, wordlist" not set  
  
(sameena@kali)-[~]  
$ ls /usr/share/wordlists/dirb/  
big.txt common.txt extensions_common.txt mutations_common.txt small.txt stress  
catala.txt euskera.txt indexes.txt others spanish.txt vulns  
  
(sameena@kali)-[~]  
$ gobuster dir -u http://testasp.vulnweb.com/ -w /usr/share/wordlists/dirb/common.txt  
=====
```

```
Gobuster v3.8  
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)  
=====
```

```
[+] Url: http://testasp.vulnweb.com/  
[+] Method: GET  
[+] Threads: 10  
[+] Wordlist: /usr/share/wordlists/dirb/common.txt  
[+] Negative Status codes: 404  
[+] User Agent: gobuster/3.8  
[+] Timeout: 10s  
=====
```

```
Starting gobuster in directory enumeration mode  
=====
```

```
/_vti_cnf (Status: 301) [Size: 159] [--> http://testasp.vulnweb.com/_vti_cnf/]  
/aspnet_client (Status: 301) [Size: 164] [--> http://testasp.vulnweb.com/aspnet_client/]  
/avatars (Status: 301) [Size: 158] [--> http://testasp.vulnweb.com/avatars/]  
/cgi-bin (Status: 301) [Size: 158] [--> http://testasp.vulnweb.com/cgi-bin/]  
/cgi-bin/ (Status: 403) [Size: 1233]  
/html (Status: 301) [Size: 155] [--> http://testasp.vulnweb.com/html/]  
/HTML (Status: 301) [Size: 155] [--> http://testasp.vulnweb.com/HTML/]  
/images (Status: 301) [Size: 157] [--> http://testasp.vulnweb.com/images/]  
/Images (Status: 301) [Size: 157] [--> http://testasp.vulnweb.com/Images/]  
/jscripts (Status: 301) [Size: 159] [--> http://testasp.vulnweb.com/jscripts/]  
/robots.txt (Status: 200) [Size: 13]  
/t (Status: 301) [Size: 152] [--> http://testasp.vulnweb.com/t/]  
/T (Status: 301) [Size: 152] [--> http://testasp.vulnweb.com/T/]  
/templates (Status: 301) [Size: 160] [--> http://testasp.vulnweb.com/templates/]  
Progress: 4613 / 4613 (100.00%)  
=====
```

```
Finished  
=====
```



Discovered Resources

- /_vti_cnf/
- /aspnet_client/
- /avatars/
- /cgi-bin/
- /html/
- /HTML/
- /images/
- /Images/
- /jscripts/
- /robots.txt
- /t/
- /T/
- /templates/

The /cgi-bin/ directory returned 403 Forbidden, which indicates that the resource exists but access is restricted.

The /robots.txt file returned 200 OK

Finding:

- Gobuster discovered several directories and files that were not directly visible from the main webpage.
- The discovered resources provided additional information about the structure and technologies of the web application and can be useful for further authorized security testing.



Disearch

It is a web path scanner used to discover hidden directories and files on a web server.

The command used: `dirsearch -u http://testasp.vulnweb.com/ -w /usr/share/wordlists/dirb/common.txt`

Discovered Resources

- `/_vti_cnf/`
- `/aspnet_client/`
- `/avatars/`
- `/cgi-bin/`
- `/HTML/`
- `/html/`
- `/images/`
- `/Images/`
- `/jscripts/`
- `/robots.txt`
- `/T/`
- `/t/`
- `/templates/`

The `/cgi-bin/` directory returned `*403 Forbidden`, while `/robots.txt` returned `**200 OK`

Finding:

The results were mostly consistent with the Gobuster scan. This increased confidence that the discovered resources were actually present on the target.



```
Apps Places Sep 26 22:42 2% 38% ↑0.0 kB ↓0.0 kB sameena@kali: ~

Proxy authentication credential
--replay-proxy=PROXY Proxy to replay with found paths
--tor Use Tor network as proxy
--scheme=SCHEME Scheme for raw request or if there is no scheme in the
URL (Default: auto-detect)
--max-rate=RATE Max requests per second
--retries=RETRIES Number of retries for failed requests
--ip=IP Server IP address

Advanced Settings:
--crawl Crawl for new paths in responses

View Settings:
--full-url Full URLs in the output (enabled automatically in
quiet mode)
--redirects-history Show redirects history
--no-color No colored output
-q, --quiet-mode Quiet mode

Output Settings:
-o PATH, --output=PATH Output file
--format=FORMAT Report format (Available: simple, plain, json, xml,
md, csv, html, sqlite)
--log=PATH Log file

(sameena@kali)-[~]
$ dirsearch -u http://testasp.vulnweb.com/ -w /usr/share/wordlists/dirb/common.txt
/usr/lib/python3/dist-packages/dirsearch/dirsearch.py:23: DeprecationWarning: pkg_resources is deprecated as an API. See https://set
uptools.pypa.io/en/latest/pkg_resources.html
from pkg_resources import DistributionNotFound, VersionConflict

dirsearch v0.4.3

Extensions: php, aspx, jsp, html, js | HTTP method: GET | Threads: 25 | Wordlist size: 4613

Output File: /home/sameena/reports/http_testasp.vulnweb.com/__26-09-26_22-17-25.txt

Target: http://testasp.vulnweb.com/

[22:17:25] Starting:
[22:17:33] 301 - 159B - /_vti_cnf -> http://testasp.vulnweb.com/_vti_cnf/
[22:17:38] 301 - 164B - /aspnet_client -> http://testasp.vulnweb.com/aspnet_client/
[22:17:38] 301 - 158B - /avatars -> http://testasp.vulnweb.com/avatars/
[22:17:42] 301 - 158B - /cgi-bin -> http://testasp.vulnweb.com/cgi-bin/
[22:17:42] 403 - 1KB - /cgi-bin/
[22:17:55] 301 - 155B - /HTML -> http://testasp.vulnweb.com/HTML/
[22:17:55] 301 - 155B - /html -> http://testasp.vulnweb.com/html/
[22:17:56] 301 - 157B - /images -> http://testasp.vulnweb.com/images/
[22:17:56] 301 - 157B - /Images -> http://testasp.vulnweb.com/Images/
[22:17:59] 301 - 159B - /jscripts -> http://testasp.vulnweb.com/jscripts/
[22:18:15] 200 - 13B - /robots.txt
[22:18:22] 301 - 152B - /T -> http://testasp.vulnweb.com/T/
[22:18:23] 301 - 152B - /t -> http://testasp.vulnweb.com/t/
[22:18:23] 301 - 160B - /templates -> http://testasp.vulnweb.com/templates/

Task Completed

(sameena@kali)-[~]
$
```



Dirb Enumeration

The **command used**: `dirb http://testasp.vulnweb.com/ /usr/share/dirb/wordlists/common.txt -z 100`

The `-z 100` option added a 100 millisecond delay between requests.

Resources Discovered by Dirb

- `/_vti_cnf/`
- `/aspnet_client/`
- `/avatars/`
- `/cgi-bin/`
- `/html/`
- `/HTML/`
- `/images/`
- `/Images/`
- `/jscripts/`
- `/robots.txt`
- `/t/`
- `/T/`
- `/templates/`

The main scan successfully identified the resources listed above. Therefore, the Dirb results were used as supporting evidence and were compared with the completed Gobuster and Dirsearch results



Finding 1: Directory Information Disclosure

Several directories were discoverable through automated enumeration.

Impact:

Attackers may obtain additional information about the structure of the web application, which can help with further reconnaissance.

Recommendation:

Only required directories should be publicly accessible, and unnecessary resources should be removed or properly restricted.

Finding 2: Restricted /cgi-bin/ Directory

The /cgi-bin/ directory was discovered but returned *403 Forbidden*.

Impact:

The existence of the directory is visible even though access is restricted.

Recommendation:

Keeping unnecessary directories disabled and ensuring proper access controls are applied.

Finding 3: Publicly Accessible robots.txt

The /robots.txt file returned *200 OK*.

Impact:

Although robots.txt does not provide security protection, information contained in it can sometimes reveal paths or website structure.

Recommendation:

We should not place sensitive information in robots.



```
sameena@kali: ~  
$ dirb  
-----  
DIRB v2.22  
By The Dark Raver  
-----  
dirb <url_base> [<wordlist_file(s)>] [options]  
===== NOTES =====  
<url_base> : Base URL to scan. (Use -resume for session resuming)  
<wordlist_file(s)> : List of wordfiles. (wordfile1,wordfile2,wordfile3...)  
===== HOTKEYS =====  
'n' -> Go to next directory.  
'q' -> Stop scan. (Saving state for resume)  
'r' -> Remaining scan stats.  
===== OPTIONS =====  
-a <agent_string> : Specify your custom USER_AGENT.  
-b : Use path as is.  
-c <cookie_string> : Set a cookie for the HTTP request.  
-E <certificate> : path to the client certificate.  
-f : Fine tuning of NOT_FOUND (404) detection.  
-H <header_string> : Add a custom header to the HTTP request.  
-i : Use case-insensitive search.  
-l : Print "Location" header when found.  
-N <nf_code>: Ignore responses with this HTTP code.  
-o <output_file> : Save output to disk.  
-p <proxy[:port]> : Use this proxy. (Default port is 1080)  
-P <proxy_username:proxy_password> : Proxy Authentication.  
-r : Don't search recursively.  
-R : Interactive recursion. (Asks for each directory)  
-S : Silent Mode. Don't show tested words. (For dumb terminals)  
-t : Don't force an ending '/' on URLs.  
-u <username:password> : HTTP Authentication.  
-v : Show also NOT_FOUND pages.  
-w : Don't stop on WARNING messages.  
-X <extensions> / -x <exts_file> : Append each word with this extensions.  
-z <millisecs> : Add a milliseconds delay to not cause excessive Flood.  
===== EXAMPLES =====  
dirb http://url/directory/ (Simple Test)  
dirb http://url/ -X .html (Test files with '.html' extension)  
dirb http://url/ /usr/share/dirb/wordlists/vulns/apache.txt (Test with apache.txt wordlist)  
dirb https://secure_url/ (Simple Test with SSL)  
sameena@kali: ~  
$ ls /usr/share/dirb/wordlists/  
big.txt      common.txt  extensions_common.txt  mutations_common.txt  small.txt  stress  
catala.txt   euskera.txt  indexes.txt           others                 spanish.txt  vulns  
sameena@kali: ~  
$ dirb http://testasp.vulnweb.com/ /usr/share/dirb/wordlists/common.txt  
-----  
DIRB v2.22  
By The Dark Raver  
-----  
START_TIME: Sat Sep 26 22:47:51 2026  
URL BASE: http://testasp.vulnweb.com/
```



```
sameena@kali: ~  
$ ls /usr/share/dirb/wordlists/  
big.txt      common.txt  extensions_common.txt  mutations_common.txt  small.txt  stress  
catala.txt   euskera.txt  indexes.txt           others                spanish.txt vulns  
  
$ dirb http://testasp.vulnweb.com/ /usr/share/dirb/wordlists/common.txt  
  
-----  
DIRB v2.22  
By The Dark Raver  
-----  
  
START_TIME: Sat Sep 26 22:47:51 2026  
URL_BASE: http://testasp.vulnweb.com/  
WORDLIST_FILES: /usr/share/dirb/wordlists/common.txt  
  
-----  
GENERATED WORDS: 4612  
  
---- Scanning URL: http://testasp.vulnweb.com/ ----  
==> DIRECTORY: http://testasp.vulnweb.com/_vti_cnf/  
==> DIRECTORY: http://testasp.vulnweb.com/aspnet_client/  
==> DIRECTORY: http://testasp.vulnweb.com/avatars/  
==> DIRECTORY: http://testasp.vulnweb.com/cgi-bin/  
+ http://testasp.vulnweb.com/cgi-bin/ (CODE:403|SIZE:1233)  
  
==> DIRECTORY: http://testasp.vulnweb.com/html/  
==> DIRECTORY: http://testasp.vulnweb.com/HTML/  
==> DIRECTORY: http://testasp.vulnweb.com/images/  
==> DIRECTORY: http://testasp.vulnweb.com/Images/  
==> DIRECTORY: http://testasp.vulnweb.com/jscripts/  
+ http://testasp.vulnweb.com/robots.txt (CODE:200|SIZE:13)  
^C> Testing: http://testasp.vulnweb.com/scheduled  
  
$ ls -l  
total 40  
drwxr-xr-x 2 sameena sameena 4096 Feb 13 2026 Desktop  
drwxr-xr-x 2 sameena sameena 4096 Dec 20 2025 Documents  
drwxr-xr-x 2 sameena sameena 4096 Dec 20 2025 Downloads  
drwxrwxr-x 2 sameena sameena 4096 Aug 12 20:08 linuxpractice  
drwxr-xr-x 2 sameena sameena 4096 Dec 20 2025 Music  
drwxr-xr-x 3 sameena sameena 4096 Sep 25 21:27 Pictures  
drwxr-xr-x 2 sameena sameena 4096 Dec 20 2025 Public  
drwxrwxr-x 3 sameena sameena 4096 Sep 26 22:17 reports  
drwxr-xr-x 2 sameena sameena 4096 Dec 20 2025 Templates  
drwxr-xr-x 2 sameena sameena 4096 Dec 20 2025 Videos  
  
$
```



```

Apps  Places  Sep 27 20:37  2%  32%  ↑0.0 kB  ↓0.0 kB  sameena@kali: ~

[sameena@kali]~$ dirb http://testasp.vulnweb.com/ /usr/share/dirb/wordlists/common.txt -z 100~

-----
DIRB v2.22
By The Dark Raver
-----

START_TIME: Sun Sep 27 19:55:44 2026
URL_BASE: http://testasp.vulnweb.com/
WORDLIST_FILES: /usr/share/dirb/wordlists/common.txt
SPEED_DELAY: 100 milliseconds

-----
GENERATED WORDS: 4612

---- Scanning URL: http://testasp.vulnweb.com/ ----
==> DIRECTORY: http://testasp.vulnweb.com/_vti_cnf/
==> DIRECTORY: http://testasp.vulnweb.com/aspnet_client/
==> DIRECTORY: http://testasp.vulnweb.com/avatars/
==> DIRECTORY: http://testasp.vulnweb.com/cgi-bin/
+ http://testasp.vulnweb.com/cgi-bin/ (CODE:403|SIZE:1233)
==> DIRECTORY: http://testasp.vulnweb.com/html/
==> DIRECTORY: http://testasp.vulnweb.com/HTML/
==> DIRECTORY: http://testasp.vulnweb.com/images/
==> DIRECTORY: http://testasp.vulnweb.com/Images/
==> DIRECTORY: http://testasp.vulnweb.com/jscripts/
+ http://testasp.vulnweb.com/robots.txt (CODE:200|SIZE:13)
==> DIRECTORY: http://testasp.vulnweb.com/t/
==> DIRECTORY: http://testasp.vulnweb.com/T/
==> DIRECTORY: http://testasp.vulnweb.com/templates/

---- Entering directory: http://testasp.vulnweb.com/_vti_cnf/ ----

(!) FATAL: Too many errors connecting to host
(Possible cause: COULDNT RESOLVE HOST)

-----
END_TIME: Sun Sep 27 20:35:44 2026
DOWNLOADED: 4897 - FOUND: 2

[sameena@kali]~$
```



PROJECT 3

LOGIN TRAFFIC ANALYSIS

1. Introduction

Login pages handle sensitive information such as usernames and passwords. If this information is transmitted without encryption, it may be exposed to attackers who can monitor the network traffic.

In this project, I captured and analyzed the network traffic generated during a login session on the authorized vulnerable website **testasp.vulnweb.com**. Wireshark was used to inspect the login request and determine whether the credentials were transmitted securely.

2. Objective

The main objectives of this project were:

- To capture network traffic during a login session.
- To identify the login request generated by the application.
- To examine how the username and password were transmitted.
- To determine whether the login information was protected using encryption.
- To document the security weakness identified during the analysis.

3. Target

Target: <http://testasp.vulnweb.com/>

4. Tools Used

- Wireshark – Used to capture and analyze network packets.
- Burp Suite Community Edition – Opened for additional HTTP traffic analysis.



5. Methodology

The following steps were followed:

1. Opened Wireshark and started packet capture on the active eth0 interface.
2. Opened the target website.
3. Navigated to the login page.
4. Used dummy test credentials to generate a login request.
5. Stopped or analyzed the captured traffic in Wireshark.
6. Applied the http display filter.
7. Located the POST request to /Login.asp.
8. Expanded the HTTP details and inspected the form fields.
9. Observed that the username and password fields were present in the HTTP request.
10. Analyzed the security of the login transmission.

Wireshark Traffic Analysis

After performing the login attempt, Wireshark was used to identify the HTTP traffic.

The HTTP filter was applied:

http

A POST request to the login endpoint was identified:

POST /Login.asp

The HTTP details showed that the request contained form fields for the username and password.

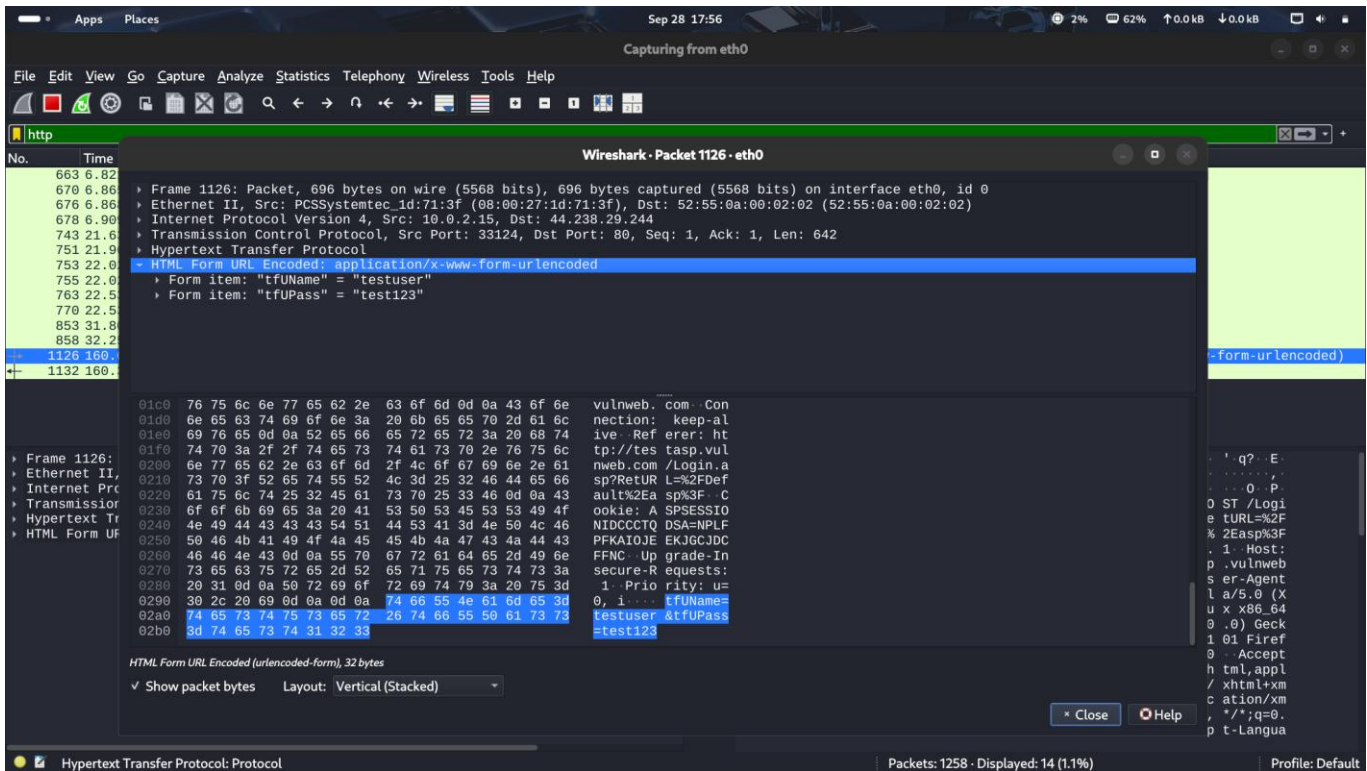


Fig: HTTP Request

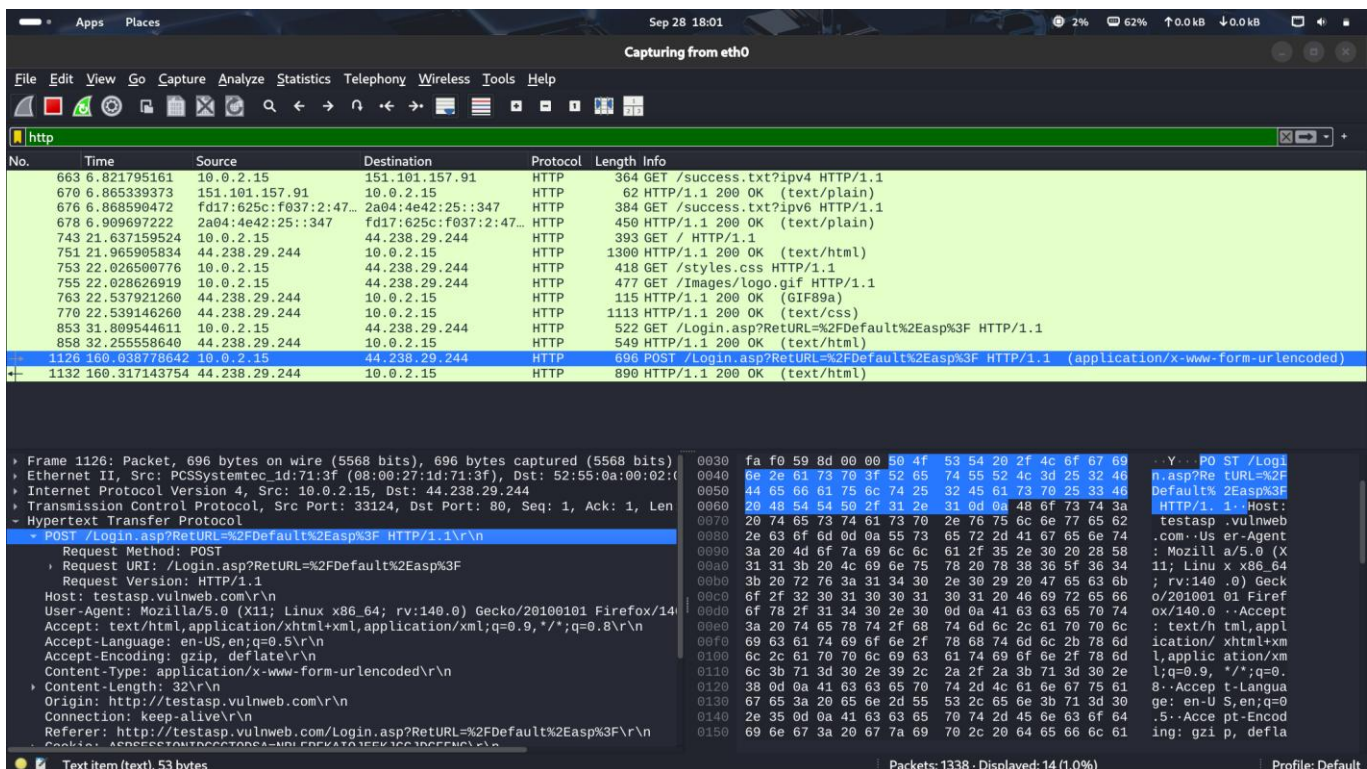


Fig: Login Form Fields



Finding

- The login request was submitted using the **HTTP** protocol through a POST request to /Login.asp.
- During Wireshark analysis, the submitted form contained username and password fields.
- Because HTTP does not provide TLS encryption, sensitive login information transmitted through HTTP can potentially be observed by someone who is able to monitor the network traffic.

Evidence

The captured packet showed:

- Request Method: POST
- Request URI: /Login.asp
- Protocol: HTTP
- Form fields: Username and Password

Risk Level: **High**

Conclusion

In this project, I captured and analyzed the network traffic generated during a login session on the authorized vulnerable website.

Wireshark was used to identify the POST request to /Login.asp and inspect the submitted form fields.

The analysis showed that the login information was transmitted using HTTP rather than HTTPS. This means the communication was not protected by TLS encryption and could potentially expose sensitive information if the network traffic were monitored.



PROJECT 4

BASIC WEB VULNERABILITY ASSESSMENT

1. Introduction

Web applications may contain security weaknesses that can expose sensitive information or make the application more vulnerable to attacks. A basic vulnerability assessment helps identify these weaknesses and understand their possible security impact.

In this project, I performed a basic security assessment on the authorized vulnerable website **testasp.vulnweb.com**. The main purpose was to identify common web security issues, classify their severity, and understand suitable mitigation techniques.

For this assessment, I used **Nikto** and **Burp Suite Community Edition**. The findings and screenshots were recorded as part of the project documentation.

2. Objective

The main objectives of this project were:

- To perform a basic security assessment of a vulnerable web application.
- To identify common web security weaknesses.
- To understand the severity and possible impact of the findings.
- To use Nikto and Burp Suite for security testing.
- To document the findings with supporting screenshots.
- To provide suitable mitigation recommendations.



3. Target

Target Website:

<http://testasp.vulnweb.com/>

Target IP:

44.238.29.244

4. Tools Used

- **Nikto v2.5.0** – Used to identify common web server and configuration issues.
- **Burp Suite Community Edition** – Used to capture and inspect HTTP requests.

5. Methodology

The assessment was performed in the following steps:

1. Accessed the authorized vulnerable website.
2. Performed a basic web server security scan using Nikto.
3. Recorded the security-related findings reported by Nikto.
4. Opened Burp Suite and used its built-in browser.
5. Accessed the target website through Burp Suite.
6. Captured the HTTP GET request generated by the browser.
7. Checked the request and HTTP response status in Burp's HTTP history.
8. Recorded screenshots as supporting evidence.
9. Classified the identified issues and prepared mitigation recommendations.



Nikto Scan

The following command was used:

```
nikto -h http://testasp.vulnweb.com/
```

Nikto identified several security-related issues in the target web application.

Vulnerability Findings

Finding 1: Missing HttpOnly Cookie Flag

Severity: Medium

Nikto reported that the ASPSESSIONID cookie was created without the HttpOnly flag.

This means the session cookie may be accessible through client-side scripts. If an attacker is able to execute malicious JavaScript in the application, the cookie could potentially be exposed.

Evidence:

Cookie ASPSESSIONIDCCCTQDSA created without the httponly flag.

Finding 2: Missing X-Frame-Options Header

Severity: Medium

Nikto reported that the X-Frame-Options security header was not present.

This header can help prevent a webpage from being loaded inside a frame on another website. Its absence can increase the risk of clickjacking attacks.

Evidence:

The anti-clickjacking X-Frame-Options header is not present.



Finding 3: Missing X-Content-Type-Options Header

Severity: Low

Nikto reported that the X-Content-Type-Options header was not set.

This security header helps prevent browsers from incorrectly interpreting the content type of a response.

Evidence:

The X-Content-Type-Options header is not set.

Finding 4: Internal IP Address Disclosure

Severity: Medium

Nikto identified an internal IP address in the Location header.

The disclosed internal IP address was:

10.0.0.14

Revealing internal network information can provide additional information about the server's internal environment.

Evidence:

The web server may reveal its internal or real IP in the Location header.

The value is "10.0.0.14".



Burp Suite Analysis

After the Nikto scan, I used Burp Suite Community Edition to capture the HTTP traffic generated while accessing the target website.

The Burp built-in browser was used to access:

http://testasp.vulnweb.com/

Burp successfully captured the following request:

GET / HTTP/1.1

Host: testasp.vulnweb.com

- The captured request also contained normal browser headers such as User-Agent, Accept, and Connection.
- The request was successfully processed by the target, and Burp's HTTP history showed an **HTTP 200** status code.
- This confirmed that Burp Suite was successfully receiving and analyzing traffic from the authorized target.

```
sameena@kali: ~  
$ ip addr  
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000  
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00  
    inet 127.0.0.1/8 scope host lo  
        valid_lft forever preferred_lft forever  
    inet6 ::1/128 scope host noprefixroute  
        valid_lft forever preferred_lft forever  
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000  
    link/ether 08:00:27:1d:71:3f brd ff:ff:ff:ff:ff:ff  
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute eth0  
        valid_lft 86168sec preferred_lft 86168sec  
    inet6 fd17:625c:f037:2:47a6:ab36:7284:a941/64 scope global temporary dynamic  
        valid_lft 86170sec preferred_lft 14170sec  
    inet6 fd17:625c:f037:2:a00:27ff:fe1d:713f/64 scope global dynamic mngtmpaddr noprefixroute  
        valid_lft 86170sec preferred_lft 14170sec  
    inet6 fe80::a00:27ff:fe1d:713f/64 scope link noprefixroute  
        valid_lft forever preferred_lft forever  
  
sameena@kali: ~  
$ nikto -h http://testasp.vulnweb.com/  
- Nikto v2.5.0  
-----  
+ Multiple IPs found: 44.238.29.244, 64:ff9b::2cee:1df4  
+ Target IP: 44.238.29.244  
+ Target Hostname: testasp.vulnweb.com  
+ Target Port: 80  
+ Start Time: 2026-09-28 18:20:07 (GMT+5.5)  
-----  
+ Server: Microsoft-IIS/8.5  
+ /: Cookie ASPSESSIONIDCCCTQDSA created without the httponly flag. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies  
+ /: Retrieved x-powered-by header: ASP.NET.  
+ /: The anti-clickjacking X-Frame-Options header is not present. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Frame-Options  
+ /: The X-Content-Type-Options header is not set. This could allow the user agent to render the content of the site in a different fashion to the MIME type. See: https://www.netsparker.com/web-vulnerability-scanner/vulnerabilities/missing-content-type-header/  
+ /BJ4mjBVC.ashx: Retrieved x-aspnet-version header: 2.0.50727.  
+ RFC-1918 /aspnet_client: IP address found in the 'location' header. The IP is "10.0.0.14". See: https://portswigger.net/kb/issues/00600300_private-ip-addresses-disclosed  
+ /aspnet_client: The web server may reveal its internal or real IP in the Location header via a request to with HTTP/1.0. The value is "10.0.0.14". See: http://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2000-0649  
+ OPTIONS: Allowed HTTP Methods: OPTIONS, TRACE, GET, HEAD, POST .  
+ OPTIONS: Public HTTP Methods: OPTIONS, TRACE, GET, HEAD, POST .  
^C  
sameena@kali: ~
```

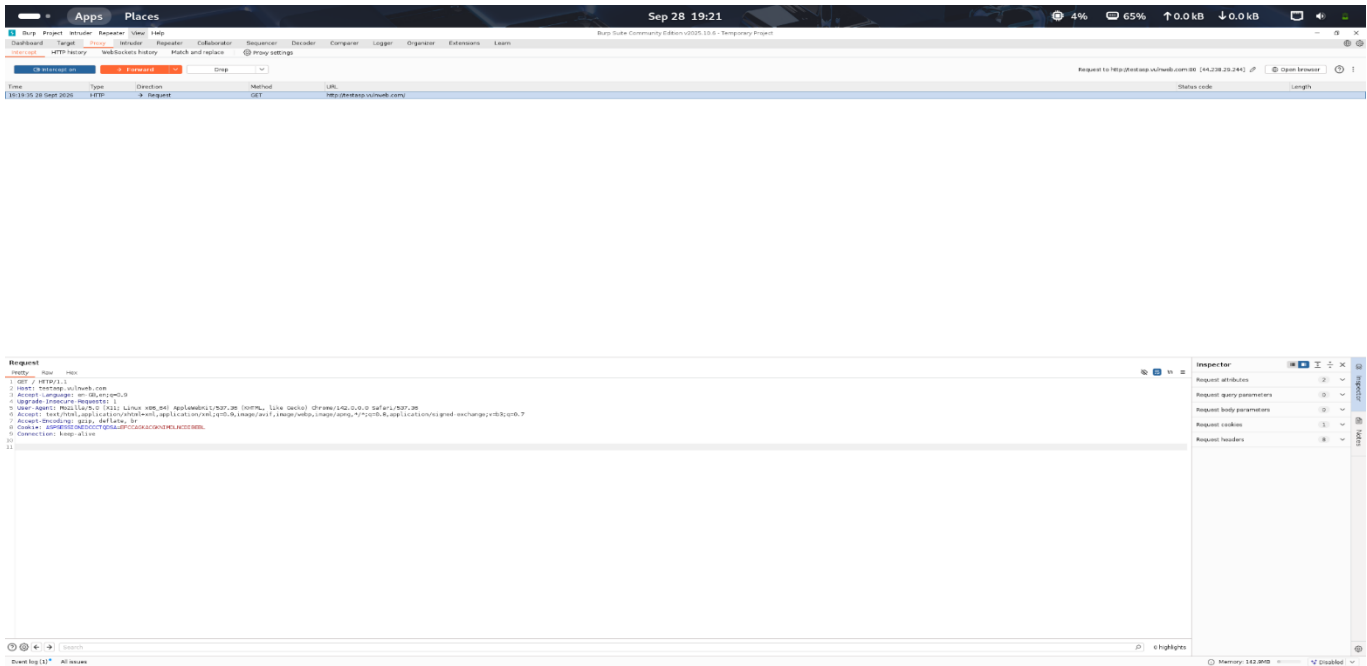


Fig: Burp Suite captured GET request for testasp.vulnweb.com.

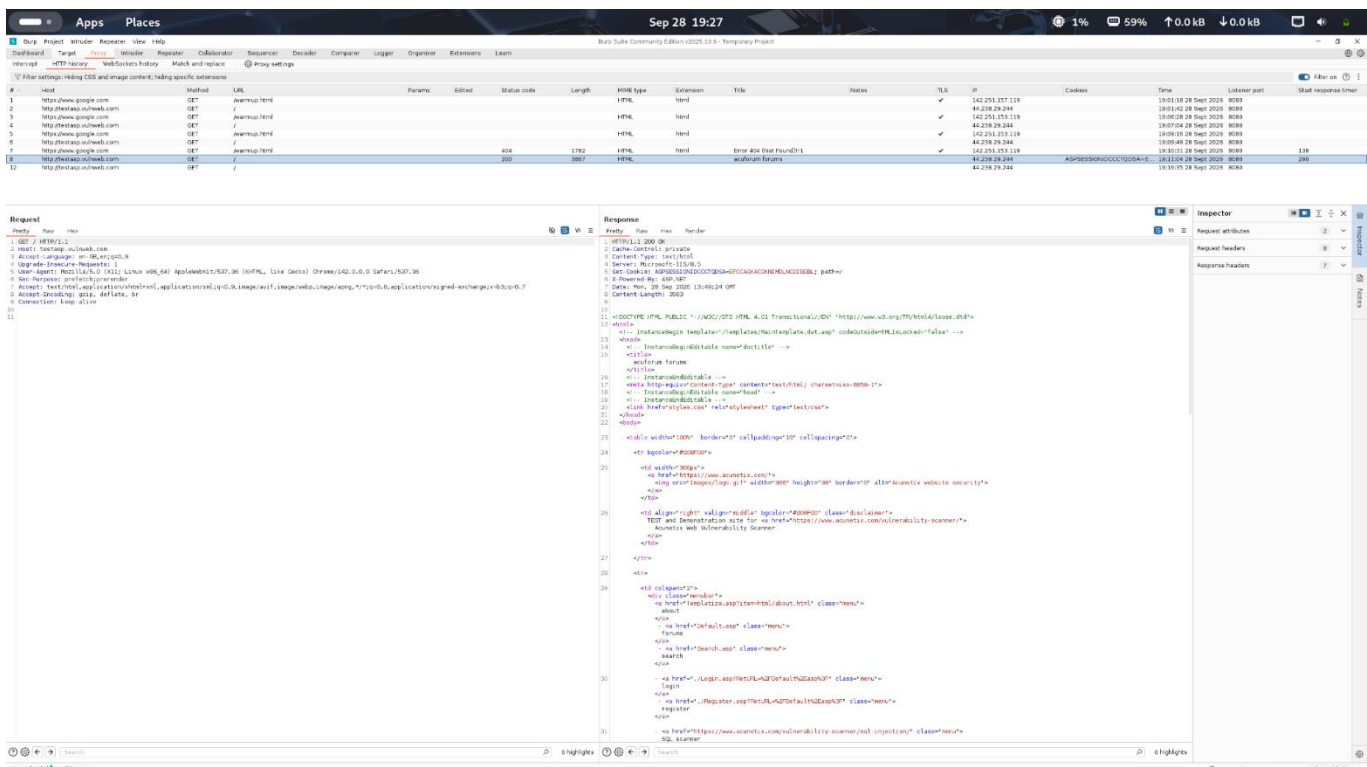


Fig: Burp Suite HTTP history showing the successful HTTP 200 response.



Conclusion

In this project, I performed a basic vulnerability assessment of the authorized vulnerable website **testasp.vulnweb.com** using Nikto and Burp Suite.

Nikto identified multiple security configuration and information disclosure issues, including missing security headers, an insecure cookie configuration, internal IP disclosure, an enabled TRACE method, and ASP.NET version disclosure.

Burp Suite was used to capture and inspect the HTTP request generated while accessing the target website. The request was successfully captured and returned an HTTP 200 response.

This project helped me understand how automated vulnerability scanning and manual HTTP traffic analysis can be used together to identify and document basic web application security issues.



ADVANCED LEVEL

PROJECT 1

COMPLETE WEB APPLICATION PENETRATION TEST

1. Introduction

Web applications can contain security vulnerabilities that may allow attackers to access sensitive information or perform unauthorized actions.

In this project, I performed a penetration test on an authorized local vulnerable web application, Damn Vulnerable Web Application (DVWA). The main purpose was to identify and demonstrate common web application vulnerabilities in a controlled environment.

The assessment included reconnaissance, service enumeration, application discovery, vulnerability testing, exploitation of confirmed vulnerabilities, risk assessment, and mitigation recommendations.

2. Objective

The main objectives of this project were:

- To perform reconnaissance on the target application.
- To identify the services running on the target.
- To discover and analyze web application vulnerabilities.
- To safely demonstrate the impact of identified vulnerabilities.
- To record screenshots and evidence of successful testing.
- To assess the risks and provide suitable mitigation recommendations.



3. Scope and Target

Target: DVWA running locally

Target URL: http://127.0.0.1:8080/

Testing Environment: Kali Linux

4. Tools Used

- Nmap – Used for service enumeration.
- DVWA – Used as the intentionally vulnerable web application for testing.

5. Methodology

The following steps were followed during the penetration test:

1. Set up the DVWA application in the local environment.
2. Performed basic reconnaissance of the target.
3. Used Nmap to identify the running service.
4. Explored the available DVWA modules.
5. Tested selected vulnerability modules.
6. Confirmed vulnerabilities using safe test inputs.
7. Captured screenshots as evidence.
8. Assessed the possible impact of the findings.
9. Prepared mitigation recommendations.



6. Reconnaissance

The target application was hosted locally at:

http://127.0.0.1:8080/

- The DVWA login page and application modules were accessible through the browser.
- The application contained several intentionally vulnerable modules such as SQL Injection, Command Injection, File Inclusion, and Cross-Site Scripting.
- The DVWA security level was set to **Low** for the testing. This setting is intentionally designed to expose vulnerable coding practices for security training.

7. Service Enumeration

Nmap was used to identify the service running on the target.

Command Used

```
nmap -sV -p 8080 127.0.0.1
```

Result

The scan showed:

```
8080/tcp open http (PHP 8.4.24)
```

This confirmed that an HTTP service was running on port 8080 and that the application was using PHP.



Screenshot: Nmap Service Enumeration

```
sameena@kali: ~  
$ curl -I http://127.0.0.1:8080/  
HTTP/1.1 302 Found  
Host: 127.0.0.1:8080  
Date: Sun, 27 Sep 2026 15:48:33 GMT  
Connection: close  
X-Powered-By: PHP/8.4.24  
Set-Cookie: security=impossible; path=/; HttpOnly  
Expires: Thu, 19 Nov 1981 08:52:00 GMT  
Cache-Control: no-store, no-cache, must-revalidate  
Pragma: no-cache  
Set-Cookie: PHPSESSID=d11660901ea0997b8e2749aaabc1f0e6; expires=Mon, 28 Sep 2026 15:48:33 GMT; Max-Age=86400; path=/; domain=127.0.0.1; HttpOnly; SameSite=Strict  
Location: login.php  
Content-type: text/html; charset=UTF-8  
Status: 302 Found  
Server: Apache/2.4.18 (Ubuntu)  
Vulnerability: SQL Injection  
More Information  
1 service unrecognized despite returning data. If you know the service/version, please submit the following fingerprint at https://nmap.org/cgi-bin/submit.cgi?new-service :  
SF-Port8080-TCP:V=7.95I=7%0=9/27Time=6AB93C70%P=x86_64-pc-linux-gnu%r(Ge  
SF:trRequest,1D7,"HTTP/1.0x20302x20Foundr\ndate:\x20Sun,\x2027\x20Sep\x  
SF:202026\x2015:55:28\x20GMT\r\nConnection:\x20close\r\nX-Powered-By:\x20P  
SF:HP/8\4\24\r\nSet-Cookie:\x20security=impossible;\x20path=/;\x20HttpOnly  
SF:ly\r\nExpires:\x20Thu,\x2019\x20Nov\x201981\x2008:52:00\x20GMT\r\nCache  
SF:-Control:\x20no-store,\x20no-cache,\x20must-revalidate\r\nPragma:\x20no  
SF:-cache\r\nSet-Cookie:\x20PHPSESSID=83ec827c334437e1e8c47c42a389b813;\x2  
SF:0expires=Mon,\x2028\x20Sep\x202026\x2015:55:28\x20GMT;\x20Max-Age=86400  
SF:\x20path=/;\x20HttpOnly;\x20SameSite=Strict\r\nLocation:\x20login.php  
SF:r\nContent-type:\x20text/html;\x20charset=UTF-8\r\n\r\n")%r(HTTPOption  
SF:s,1D7,"HTTP/1.0x20302x20Foundr\ndate:\x20Sun,\x2027\x20Sep\x202026\x  
SF:2015:55:28\x20GMT\r\nConnection:\x20close\r\nX-Powered-By:\x20PHP/8\4  
SF:\24\r\nSet-Cookie:\x20security=impossible;\x20path=/;\x20HttpOnly\r\nE  
SF:pires:\x20Thu,\x2019\x20Nov\x201981\x2008:52:00\x20GMT\r\nCache-Contro  
SF:l:\x20no-store,\x20no-cache,\x20must-revalidate\r\nPragma:\x20no-cache\  
SF:r\nSet-Cookie:\x20PHPSESSID=ba594975de30d069803e815fb3ffc32b;\x20expire  
SF:s=Mon,\x2028\x20Sep\x202026\x2015:55:28\x20GMT;\x20Max-Age=86400;\x20pa  
SF:th=/;\x20HttpOnly;\x20SameSite=Strict\r\nLocation:\x20login.php\r\nCon  
SF:tent-type:\x20text/html;\x20charset=UTF-8\r\n\r\n")%r(FourOhFourRequest
```

```
sameena@kali: ~  
$ nmap -sV -p 8080 127.0.0.1  
Starting Nmap 7.95 ( https://nmap.org ) at 2026-09-27 21:25 IST  
Nmap scan report for localhost (127.0.0.1)  
Host is up (0.00014s latency).  
PORT      STATE SERVICE VERSION  
8080/tcp  open  http      (PHP 8.4.24)  
1 service unrecognized despite returning data. If you know the service/version, please submit the following fingerprint at https://nmap.org/cgi-bin/submit.cgi?new-service :  
SF-Port8080-TCP:V=7.95I=7%0=9/27Time=6AB93C70%P=x86_64-pc-linux-gnu%r(Ge  
SF:trRequest,1D7,"HTTP/1.0x20302x20Foundr\ndate:\x20Sun,\x2027\x20Sep\x  
SF:202026\x2015:55:28\x20GMT\r\nConnection:\x20close\r\nX-Powered-By:\x20P  
SF:HP/8\4\24\r\nSet-Cookie:\x20security=impossible;\x20path=/;\x20HttpOnly  
SF:ly\r\nExpires:\x20Thu,\x2019\x20Nov\x201981\x2008:52:00\x20GMT\r\nCache  
SF:-Control:\x20no-store,\x20no-cache,\x20must-revalidate\r\nPragma:\x20no  
SF:-cache\r\nSet-Cookie:\x20PHPSESSID=83ec827c334437e1e8c47c42a389b813;\x2  
SF:0expires=Mon,\x2028\x20Sep\x202026\x2015:55:28\x20GMT;\x20Max-Age=86400  
SF:\x20path=/;\x20HttpOnly;\x20SameSite=Strict\r\nLocation:\x20login.php  
SF:r\nContent-type:\x20text/html;\x20charset=UTF-8\r\n\r\n")%r(HTTPOption  
SF:s,1D7,"HTTP/1.0x20302x20Foundr\ndate:\x20Sun,\x2027\x20Sep\x202026\x  
SF:2015:55:28\x20GMT\r\nConnection:\x20close\r\nX-Powered-By:\x20PHP/8\4  
SF:\24\r\nSet-Cookie:\x20security=impossible;\x20path=/;\x20HttpOnly\r\nE  
SF:pires:\x20Thu,\x2019\x20Nov\x201981\x2008:52:00\x20GMT\r\nCache-Contro  
SF:l:\x20no-store,\x20no-cache,\x20must-revalidate\r\nPragma:\x20no-cache\  
SF:r\nSet-Cookie:\x20PHPSESSID=ba594975de30d069803e815fb3ffc32b;\x20expire  
SF:s=Mon,\x2028\x20Sep\x202026\x2015:55:28\x20GMT;\x20Max-Age=86400;\x20pa  
SF:th=/;\x20HttpOnly;\x20SameSite=Strict\r\nLocation:\x20login.php\r\nCon  
SF:tent-type:\x20text/html;\x20charset=UTF-8\r\n\r\n")%r(FourOhFourRequest  
SF:1D7,"HTTP/1.0x20302x20Foundr\ndate:\x20Sun,\x2027\x20Sep\x202026\x  
SF:2015:55:28\x20GMT\r\nConnection:\x20close\r\nX-Powered-By:\x20PHP/8\4  
SF:\24\r\nSet-Cookie:\x20security=impossible;\x20path=/;\x20HttpOnly\r\nE  
SF:pires:\x20Thu,\x2019\x20Nov\x201981\x2008:52:00\x20GMT\r\nCache-Contro  
SF:l:\x20no-store,\x20no-cache,\x20must-revalidate\r\nPragma:\x20no-cache\  
SF:r\nSet-Cookie:\x20PHPSESSID=c91fa57cdce02535df775d6f1cce9dbb;\x20expires  
SF:=Mon,\x2028\x20Sep\x202026\x2015:55:28\x20GMT;\x20Max-Age=86400;\x20pat  
SF:h=/;\x20HttpOnly;\x20SameSite=Strict\r\nLocation:\x20login.php\r\nCon  
SF:tent-type:\x20text/html;\x20charset=UTF-8\r\n\r\n")%r(OfficeScan,1D7,"HT  
SF:TP/1.1x20302x20Foundr\ndate:\x20Sun,\x2027\x20Sep\x202026\x2015:55:  
SF:41\x20GMT\r\nConnection:\x20close\r\nX-Powered-By:\x20PHP/8\4\24\r\ns  
SF:et-Cookie:\x20security=impossible;\x20path=/;\x20HttpOnly\r\nExpires:\x  
SF:20Thu,\x2019\x20Nov\x201981\x2008:52:00\x20GMT\r\nCache-Control:\x20no-  
SF:store,\x20no-cache,\x20must-revalidate\r\nPragma:\x20no-cache\r\nSet-Co  
SF:okie:\x20PHPSESSID=c96c02359e0cc27d03d3a0ae56553311;\x20expires=Mon,\x2  
SF:028\x20Sep\x202026\x2015:55:41\x20GMT;\x20Max-Age=86400;\x20path=/;\x20  
SF:HttpOnly;\x20SameSite=Strict\r\nLocation:\x20login.php\r\nContent-type  
SF:\x20text/html;\x20charset=UTF-8\r\n\r\n");  
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .  
Nmap done: 1 IP address (1 host up) scanned in 18.96 seconds
```



8. Vulnerability Assessment

During the testing, four vulnerabilities were successfully confirmed.

8.1 SQL Injection

Description

SQL Injection occurs when user input is directly used in a database query without proper protection.

Normal Input

1

The application returned one user record:

ID: 1

First name: admin

Surname: admin

Test Input

1' OR '1'='1

Result

The application returned multiple user records, including:

- admin
- Gordon Brown
- Hack Me
- Pablo Picasso
- Bob Smith

This showed that the input was able to change the behavior of the database query.



Impact

An attacker may be able to manipulate database queries and access information that should not normally be returned.

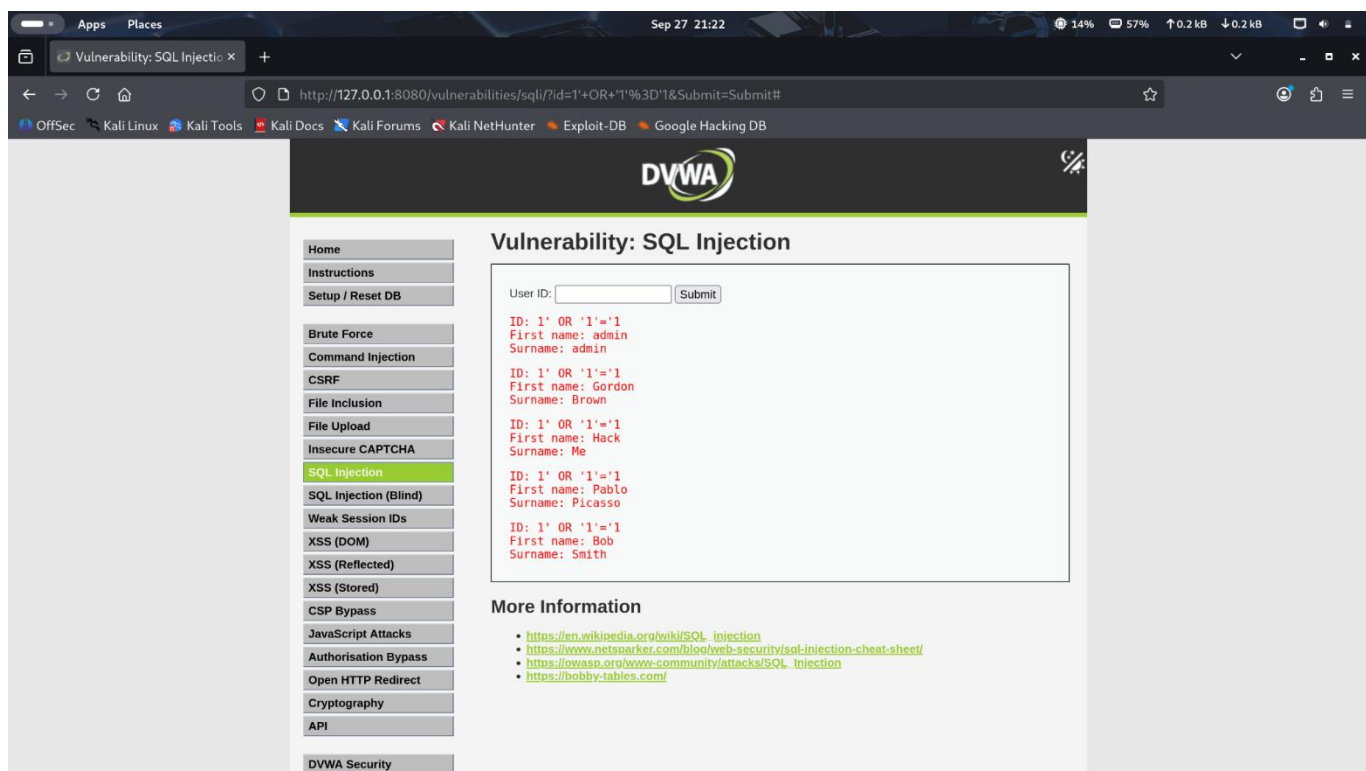
Risk Level

High

Mitigation

- Use prepared statements and parameterized queries.
- Avoid directly concatenating user input into SQL queries.
- Validate user input.
- Use database accounts with only the required permissions.

Screenshot: SQL Injection Successful Test





8.2 Reflected Cross-Site Scripting (XSS)

Description

Reflected XSS occurs when user-controlled input is reflected into a webpage without proper output encoding.

Normal Input

Sameena

Result

The application displayed:

Hello Sameena

Test Input

```
<script>alert('XSS')</script>
```

Result

A browser alert displaying **XSS** was successfully executed.

This confirmed that JavaScript supplied through the input was executed by the browser.

Impact

In a real web application, reflected XSS could allow malicious JavaScript to execute in another user's browser context. The exact impact would depend on the application's authentication and security controls.

Risk Level

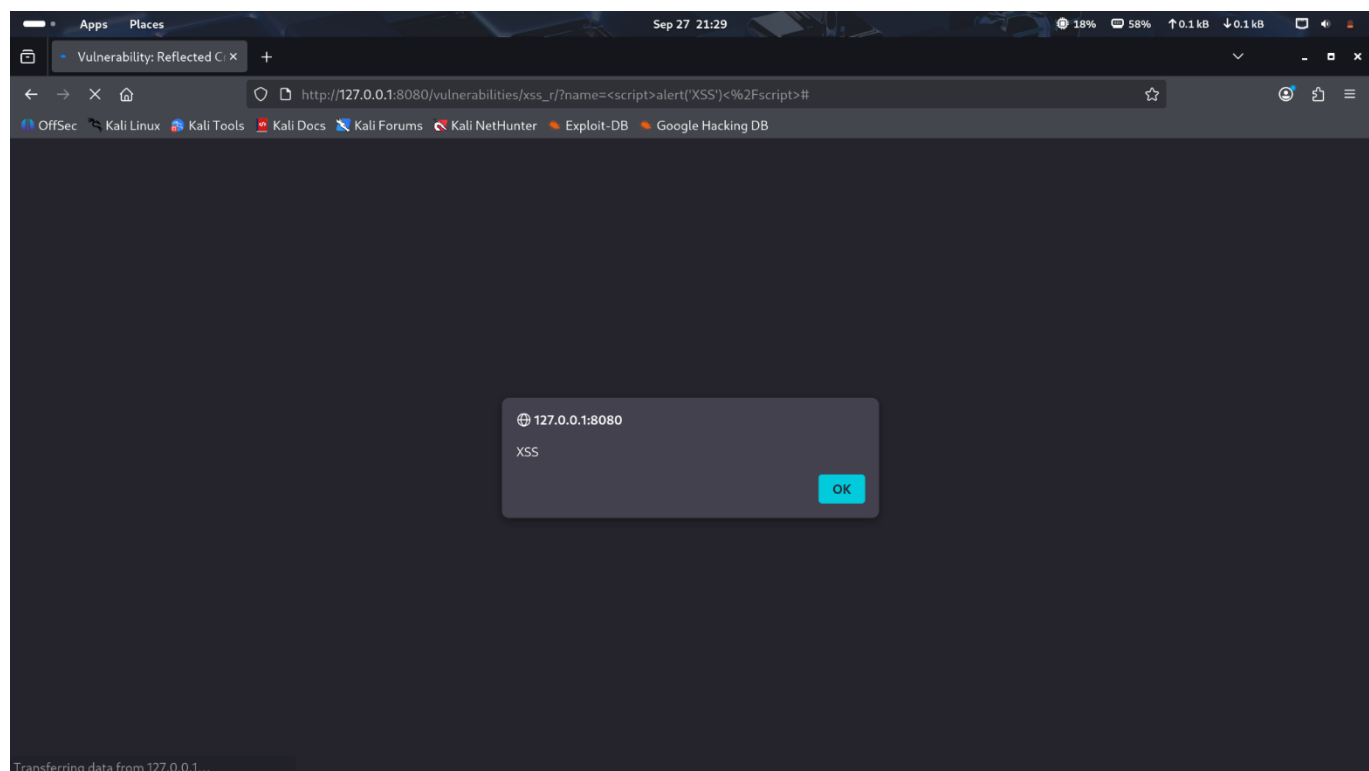
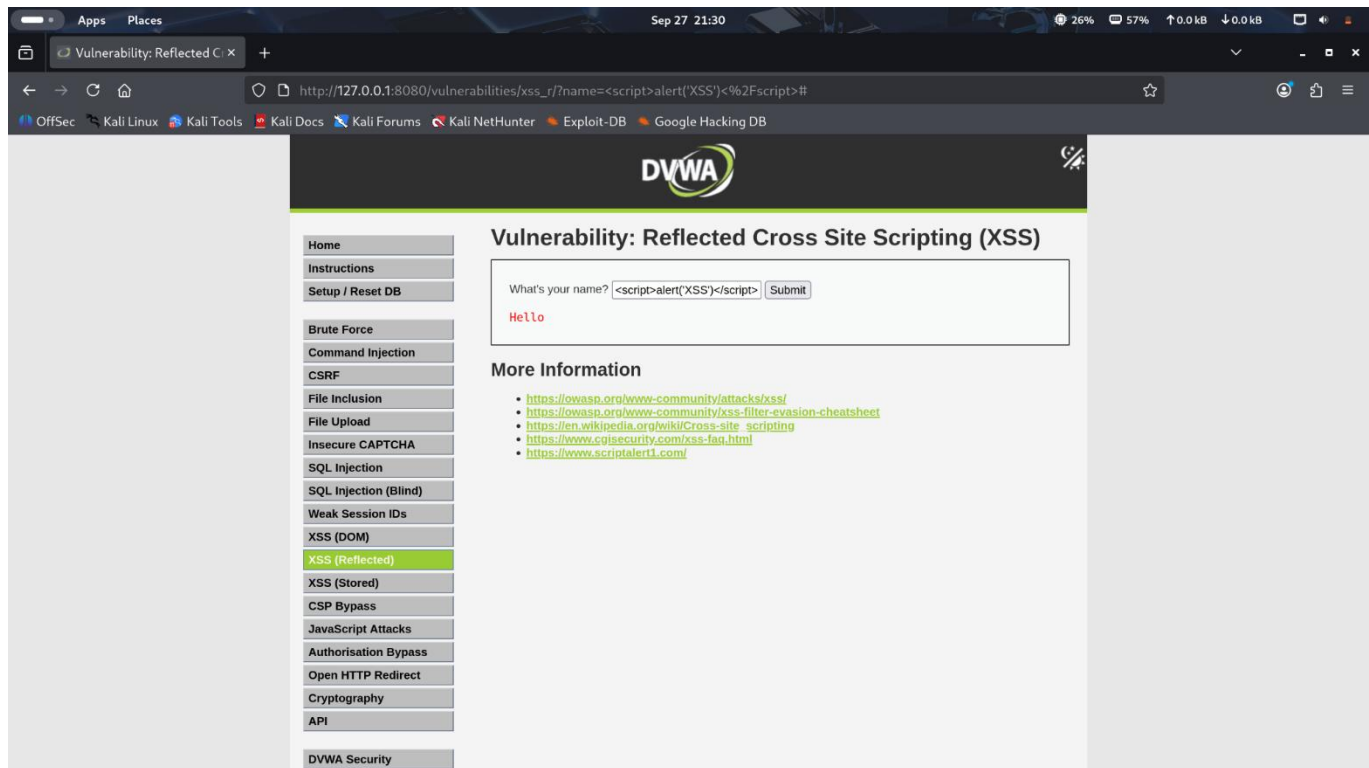
Medium

Mitigation

- Apply proper output encoding.
- Validate user input where appropriate.
- Use a suitable Content Security Policy.
- Follow secure web development practices.



Screenshot: Reflected XSS Successful Execution





8.3 Command Injection

Description

Command Injection occurs when an application passes user-controlled input to an operating-system command without proper validation.

Normal Input

127.0.0.1

The application performed a normal ping operation.

Test Input

127.0.0.1; whoami

Result

The normal ping output was displayed, followed by:

sameena

This confirmed that the additional whoami command was executed by the application.

Impact

Command Injection can potentially allow an attacker to execute operating-system commands with the privileges of the web application process.

Risk Level

Critical

Mitigation

- Avoid directly passing user input to shell commands.
- Use safer application programming interfaces instead of shell execution where possible.
- Use strict input allowlists.
- Run web applications with minimum required privileges.



Screenshot: Command Injection Successful Execution

Vulnerability: Command Injection

Ping a device

Enter an IP address:

```
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data:  
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.076 ms  
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.039 ms  
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.036 ms  
64 bytes from 127.0.0.1: icmp_seq=4 ttl=64 time=0.033 ms  
  
--- 127.0.0.1 ping statistics ---  
4 packets transmitted, 4 received, 0% packet loss, time 3075ms  
rtt min/avg/max/mdev = 0.030/0.043/0.076/0.018 ms  
sameena
```

More Information

- <https://www.scribd.com/doc/2530476/Php-Endangers-Remote-Code-Execution>
- <http://www.ss64.com/bash/>
- <http://www.ss64.com/nt/>
- https://owasp.org/www-community/attacks/Command_Injection



8.4 Local File Inclusion (LFI)

Description

Local File Inclusion occurs when an application allows a user-controlled parameter to specify a local file that is then included or displayed by the application.

Normal Input

page=file1.php

The application displayed the File 1 page.

Test Input

page=/etc/passwd

Result

The application displayed the contents of the local /etc/passwd file.

The output contained system accounts including www-data, mysql, and sameena.

This confirmed that the application allowed access to a local file through the page parameter.

Impact

An attacker may be able to read local files that are accessible to the web application process. Depending on the application and system configuration, this could expose sensitive information.

Risk Level

High

Mitigation

- Use an allowlist of permitted files or pages.
- Do not directly use user-controlled input in include or require functions.
- Validate file paths before processing them.
- Apply appropriate file permissions.



Screenshots and Evidence

The following screenshots were captured during the assessment:

- Nmap service enumeration showing port 8080 and PHP.
- SQL Injection showing multiple records after the test input.
- Reflected XSS showing successful JavaScript execution.
- Command Injection showing the whoami result.
- Local File Inclusion showing the contents of /etc/passwd.

These screenshots provide evidence for the confirmed findings.

Conclusion

In this project, I performed a complete web application penetration-testing exercise against an authorized local DVWA environment.

I performed reconnaissance and service enumeration and then tested different vulnerability modules available in the application.

During the assessment, I successfully identified and demonstrated four vulnerabilities: **SQL Injection, Reflected XSS, Command Injection, and Local File Inclusion.**

The successful tests showed how insecure input handling can affect databases, web browsers, operating-system commands, and local files.

The identified risks and mitigation recommendations were documented along with screenshots and evidence as part of the penetration-testing report.